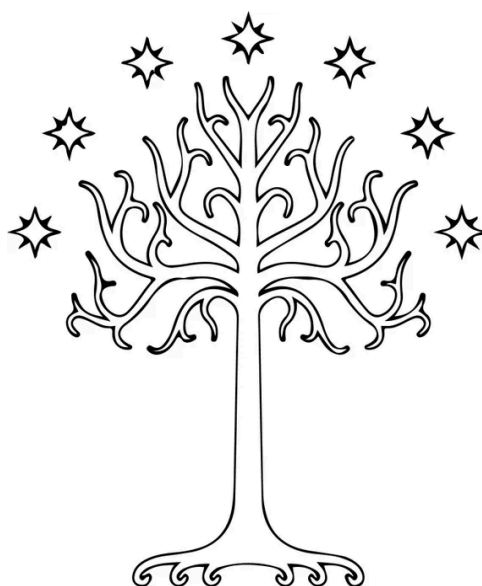




Proyectos Anteriores

ESPECIFICACIONES E IDEAS ALTERNATIVAS

7 de Marzo de 2026

1. Introducción	1
2. Especificaciones	1
2.1. Anteriores	2
2.2. Alternativas	6

1. Introducción

Se presentan las especificaciones (Stage I) de los proyectos de períodos anteriores, de modo que quiénes deseen encontrar una idea para el suyo puedan dirigir o concentrar mejor las expectativas, pero también entender que en realidad, la idea en sí no tiene límites en cuanto al dominio sobre el cual se puede desarrollar.

Es importante notar que los ejemplos de código en estas especificaciones no representan la totalidad de funcionalidades y prestaciones de cada lenguaje, sino simplemente un pequeño fragmento de algunos programas utilizados como testeo. No obstante, durante su implementación sufrieron modificaciones, y se incrementó o redujo la complejidad propuesta según corresponda, pero de todas maneras estos informes dan una idea del lenguaje y de sus prestaciones.

2. Especificaciones

Todas las especificaciones pasadas se encuentran disponibles en [📄 Stage I](#). Las ideas alternativas son aquellas que la cátedra propone, las cuáles pueden ser ampliamente modificadas o ignoradas. Solo

sirven como un puntapié para entender qué lenguajes puede que nunca se hayan realizado hasta el momento.

2.1. Anteriores

Salvo algunas excepciones, en general es notable que, a lo largo de los años, la lista de proyectos descritos en la **Tabla (2.1.1)** han incrementado significativamente su complejidad y las capacidades de los lenguajes que fueron desarrollados. En este sentido, si bien es de esperar que las ideas que el grupo seleccione puedan provenir de cualquiera de estos proyectos (o incluso de ninguno, si se trata de algo completamente novedoso), siempre se deberá hacer foco en el grado de dificultad de los proyectos más modernos (de los últimos), debido a que reflejan mucho mejor el estado actual de la cátedra.

Código	Dominio	Entrada	Salida
G-01	Imperativo. Esotérico.	Custom DSL.	C.
G-02	Probabilidad y estadística.	Custom DSL.	C.
G-03	Estructuras de datos.	Custom DSL.	Consola y C.
G-04	Data pipeline.	Custom DSL.	CSV, TSV o texto.
G-05	Orientado a objetos.	Custom DSL.	C.
G-06	Templating.	JSON aumentado.	HTML.
G-07	Templating y UI.	Custom DSL.	CSS, HTML y JavaScript.
G-08	Desarrollo de sitios web.	Custom DSL.	CSS y HTML.
G-09	Estructuras de datos.	Custom DSL.	DOT (Graphviz).
G-10	Música.	Custom DSL.	Java y JFugue.
G-11	Música.	Custom DSL.	LaTeX.
G-12	Funciones de alto orden.	Custom DSL.	Octave.
G-13	Desarrollo de sitios web.	Custom DSL.	CSS y HTML.
G-14	Dungeons & Dragons.	Custom DSL.	HTML y PDF.
G-15	UML.	Custom DSL.	Java.
G-16	Estructuras de datos.	Custom DSL.	DOT (Graphviz) y PNG.
G-17	Autómatas finitos.	Custom DSL.	Consola y C.
G-18	BPMN.	Custom DSL.	DOT (Graphviz).
G-19	Game automation.	Superset de Markdown.	JSON.
G-20	Música.	Custom DSL.	Java y JFugue.

G-21	Object-Relational Mapping.	Custom DSL.	JavaScript y Sequelize.
G-22	Algoritmos y Map-Reduce.	Superset de C.	C.
G-23	Animaciones.	Custom DSL, PNG y SVG.	HTML, JavaScript y Anime.js.
G-24	Estructuras de datos.	Custom DSL.	DOT (Graphviz).
G-25	Imágenes y física.	Custom DSL.	SVG.
G-26	Imágenes.	Custom DSL.	Python.
G-27	Infraseestructura.	Custom DSL.	XML.
G-28	Bases de datos relacionales.	Custom DSL.	SQL.
G-29	Circuitos lógicos.	Custom DSL.	C.
G-30	Imágenes.	Custom DSL.	Python.
G-31	Documentación.	Custom DSL.	HTML.
G-32	Deportes.	Subset de JSON.	HTML.
G-33	Estructuras de datos.	Custom DSL.	Python.
G-34	Electrónica.	Custom DSL.	LaTeX y CircuiTikz.
G-35	Game automation.	Custom DSL.	Python y Pygame
G-36	Diseño de modelos.	Custom DSL.	PDF.
G-37	Álgebra relacional.	SQL (PostgreSQL).	LaTeX.
G-38	Web scraping.	Custom DSL.	JavaScript y Puppeteer.
G-39	Deportes.	Subset de JSON.	HTML.
G-40	Electrónica.	Custom DSL.	LaTeX y CircuiTikz.
G-41	Blockchain.	Superset de Solidity.	Solidity.
G-42	Bases de datos relacionales.	Custom DSL.	SQL.
G-43	Testing framework.	Custom DSL.	JavaScript y Selenium.
G-44	Reverse engineering.	Custom DSL.	Python.
G-45	Documentación.	Superset de JSON.	LaTeX.
G-46	Autómatas celulares.	Custom DSL.	Python.
G-47	Música.	Custom DSL.	MIDI y PDF.
G-48	Documentación.	Custom DSL (con emojis).	LaTeX.

G-49	Planificación de procesos.	Custom DSL.	Python.
G-50	Deportes.	Custom DSL.	HTML.
G-51	Estructuras de datos.	Custom DSL.	DOT (Graphviz).
G-52	Documentación.	Superset de Markdown.	HTML.
G-53	Autómatas finitos.	Custom DSL.	LaTeX.
G-54	Game automation.	Custom DSL.	Java y libGDX.
G-55	Transpiling.	Python.	Java.
G-56	Planificación de procesos.	Custom DSL.	JSON.
G-57	Game automation.	Custom DSL.	Machination.
G-58	Deportes.	JSON.	HTML.
G-59	Bases de datos.	Custom DSL.	SQL.
G-60	Programación imperativa.	Custom DSL.	C.
G-61	Documentación.	Custom DSL.	LaTeX.
G-62	Form automation.	Custom DSL.	CSS, HTML y JavaScript.
G-63	Bases de datos relacionales.	Subset de JSON.	SQL.
G-64	Planificación de procesos.	Custom DSL.	CSS, HTML y JavaScript.
G-65	Lenguajes regulares.	Custom DSL.	DOT (Graphviz).
G-66	Analizadores léxicos.	Custom DSL.	Java.
G-67	Arte generativo.	Custom DSL.	CSS, HTML y JavaScript.
G-68	Transpiling.	TypeScript.	JavaScript.
G-69	Bases de datos relacionales.	SQL.	PlantUML.
G-70	Imágenes.	Custom DSL.	Python y Pillow.
G-71	Programación funcional.	Custom DSL.	C.
G-72	Presentaciones.	Custom DSL.	CSS, HTML y JavaScript.
G-73	Animaciones.	Custom DSL.	CSS.
G-74	Q/A games.	Superset de LaTeX.	LaTeX.
G-75	Transpiling.	Subset de C.	Assembler IA-32e.
G-76	Crochet (posta).	Custom DSL.	Python y Pillow.

G-77	Minification.	Subset de JavaScript.	JavaScript.
G-78	Documentación.	JSON.	CSS y HTML.
G-79	Desarrollo de sitios web.	Custom DSL.	CSS, HTML y JavaScript.
G-80	Documentación.	Superset de Markdown.	HTML.
G-81	Desarrollo de sitios web.	Custom DSL.	CSS y HTML.
G-82	Lógica computacional.	Custom DSL.	LaTeX.
G-83	Programación imperativa.	Custom DSL.	C.
G-84	Scheduling.	Custom DSL.	HTML.
G-85	Imágenes.	Custom DSL.	SVG.
G-86	Generación de mapas.	Custom DSL.	CSS y HTML.
G-87	Redes de información.	Custom DSL y PCAP.	C, HTML y JSON.
G-88	Game automation.	Custom DSL.	Java + libGDX.
G-89	Desarrollo de sitios web.	Subset de JSON.	HTML.
G-90	Concurrencia y paralelismo.	Custom DSL.	C.
G-91	Data pipeline.	Custom DSL.	Consola.
G-92	Bases de datos relacionales.	Subset de JSON.	SQL.
G-93	Autómatas celulares.	Custom DSL.	Python y Pygame.
G-94	Presentaciones/slides.	Markdown superset.	HTML y LaTeX Beamer.
G-95	Imágenes.	Custom DSL.	SVG.
G-96	Juegos de mesa.	Custom DSL.	Logs, métricas y reportes.
G-97	Lógica y documentación.	LaTeX.	LaTeX.
G-98	Álgebra relacional.	JSON.	SQL.
G-99	Q/A games.	Custom DSL.	HTML.
G-100	Scheduling.	Custom DSL.	HTML.
G-101	CI/CD.	Custom DSL	Makefile.
G-102	Música.	Custom DSL.	WAV y MIDI.
G-103	Data pipeline.	Custom DSL, CSV y JSON.	CSV.
G-104	Música.	Custom DSL.	MIDI.

G-105	Web y UI.	HTML.	@Composable y Kotlin
G-106	Imágenes.	Custom DSL.	BMP.
G-107	Game automation.	Subset de C.	JSON para Factorio.
G-108	Data pipeline.	Custom DSL y CSV.	HTML.
G-109	Assembler transpiler.	Assembler Z80.	Assembler x86-64.
G-110	Orientado a objetos.	Superset de C.	C.
G-111	Bases de datos relacionales.	Custom DSL.	SVG y PNG.
G-112	Electrónica.	Custom DSL.	LaTeX.
G-113	Sistemas operativos y UI.	Custom DSL.	C y PinkOS API.

Tabla 2.1.1: Una breve descripción acerca de cada lenguaje desarrollado en proyectos anteriores.

2.2. Alternativas

- (I). Desarrollar un lenguaje que permite codificar documentos en formato Latex, pero utilizando un subconjunto de sus prestaciones, limitando así el scope del proyecto, y que posea una sintaxis más amigable e intuitiva (ya que el lenguaje Latex es en general bastante críptico, y basado en macros).
- (II). Desarrollar un lenguaje que permita codificar una serie de ecuaciones y algoritmos simples, y que produzca como salida el mismo programa pero optimizado de alguna manera simple, por ejemplo, eliminado operaciones matemáticas innecesarias como sumar 0, multiplicar por 1, etc., o realizar transformaciones de código básicas, como loop-unrolling, function inlining, tail-recursion, etc.
- (III). Un minificador (*minifier*) de código para algún lenguaje de programación, o un subconjunto de algunos de ellos. El compilador debe leer dicho programa y producir en su salida un programa equivalente, pero significativamente más pequeño.
- (IV). Un traductor de lenguajes de programación, entre 2 lenguajes a elección. Por ejemplo, ingresar un programa en Java, y generar uno en Python o TypeScript. El programa de salida debe ser funcionalmente equivalente, es decir, debe operar de la misma manera en todo aspecto, o de otro modo no tendría sentido utilizar dicho compilador, ya que destruiría el programa original.
- (V). Un linter, que permita indicar mediante algún DSL las reglas activas y los archivos que debe procesar, en algún lenguaje de programación a elección. El *linter* debe analizar el código y emitir warnings y/o errores sobre el programa de entrada, aplicando las reglas configuradas. Además debe ofrecer la posibilidad de reescribir el programa para resolver la mayoría de las reglas automáticamente (dentro de lo razonable).

- (VI). Un generador de documentación, que permita sobre algún lenguaje de programación a elección, introducir comentarios con cierto formato (similar a JavaDoc, JSDoc, Swagger o Docstrings), y que entregue en su salida la documentación en formato HTML, o en PDF (ambos o alguno de ellos).